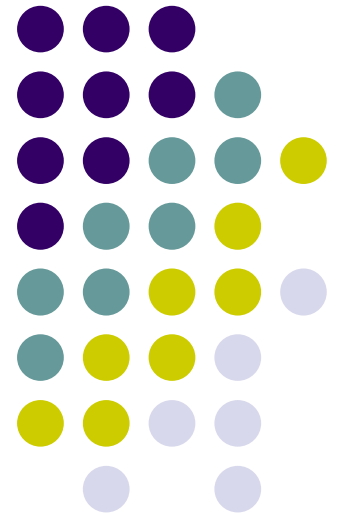
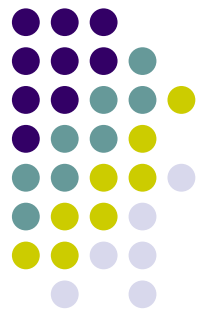


The Physical Layer





Basic concepts

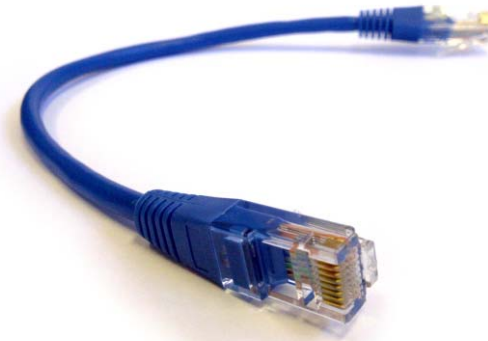
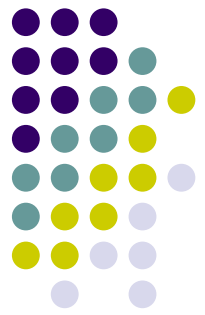
Goal: provide a bit pipe using physical phenomena

- Closely integrated with datalink layer

Today we'll look at:

- Encoding: what's a bit?
 - modulation
- Error correction: what's a correct frame?

Links - Copper



- **Copper-based Media**

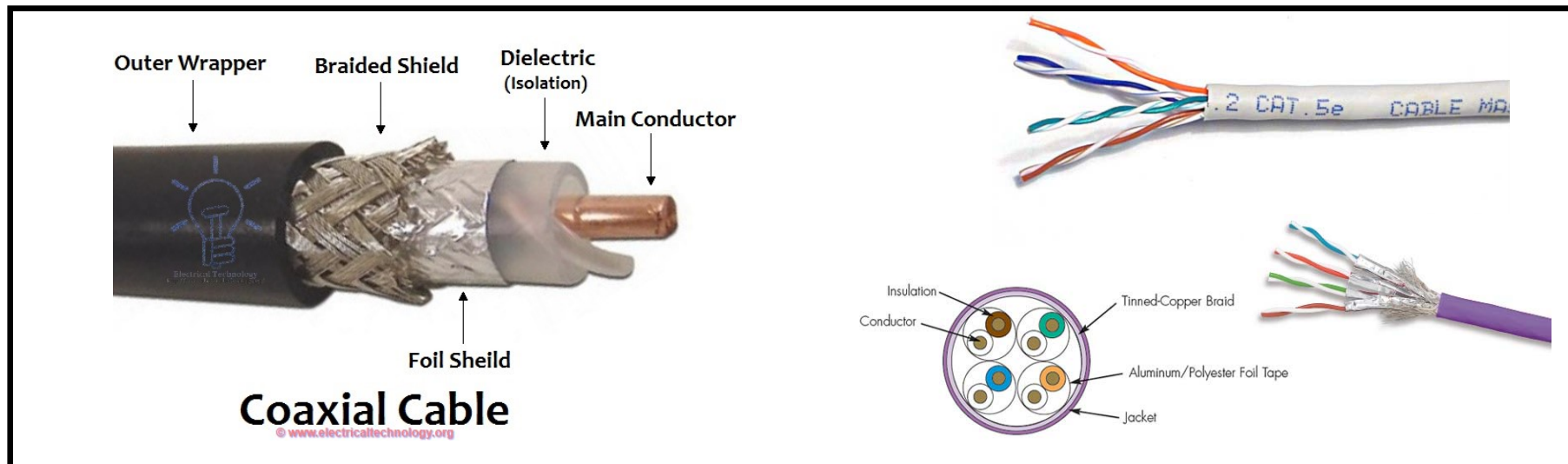
- Category 5 Twisted Pair
- Cat-7a:
- ThinNet Coaxial Cable
- ThickNet Coaxial Cable

10-100Mbps 100m

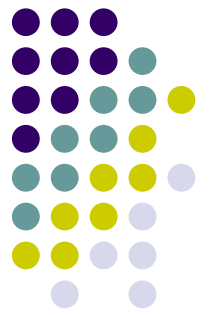
1000Mbps, 100m;

10-100Mbps 200m

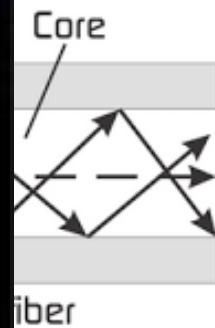
10-100Mbps 500m



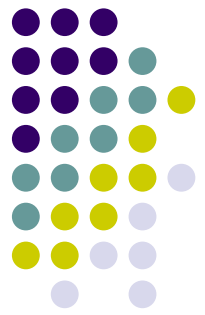
Link



- Optical
- Multimode
- Single-mode



Links - Optical



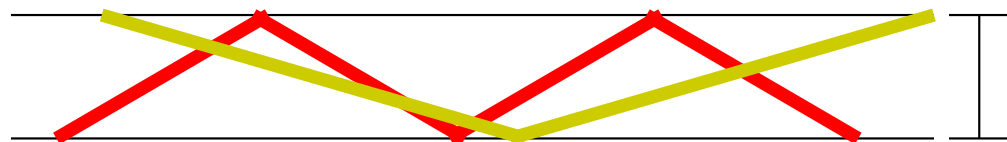
- Single mode
 - Lower attenuation (longer distances)
 - Lower dispersion (higher data rates)
- Multimode fiber
 - Cheap to drive (LED's) vs. lasers for single mode
 - Easier to terminate

core of single mode fiber



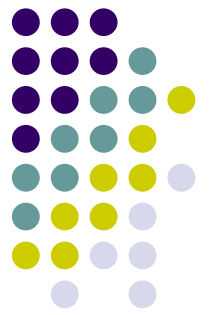
~1 wavelength thick =
~1 micron

core of multimode fiber (same frequency; colors for clarity)



O(100 microns) thick

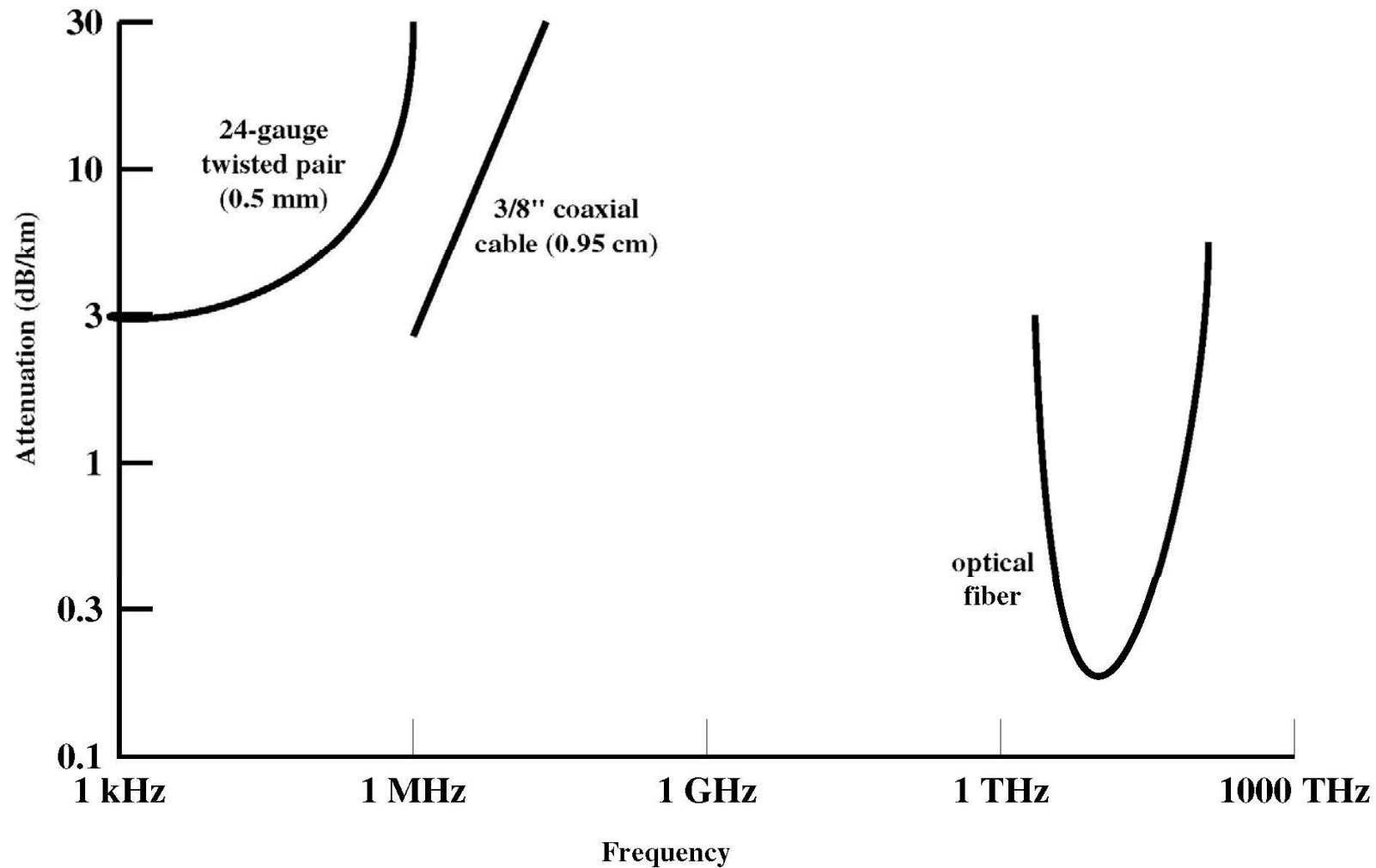
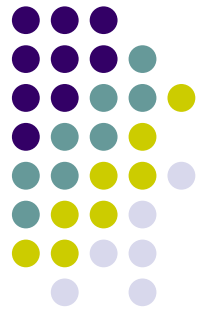
Links - Optical



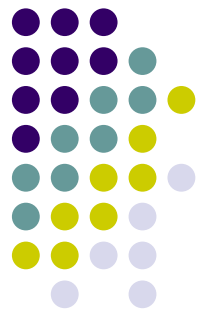
- Advantages of optical communication
 - Higher bandwidths
 - Superior attenuation properties
 - Immune from electromagnetic interference
 - No crosstalk between fibers
 - Thin, lightweight, and cheap (the fiber, not the optical-electrical interfaces)
- Disadvantages:
 - Fragile
 - Hard to tap



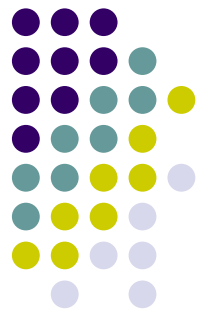
Comparison of Attenuations



Standard Lines



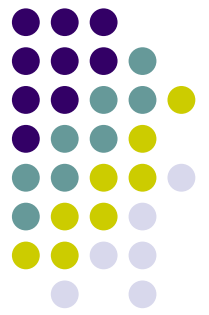
- POTS 64Kbps
- ADSL 24Mbps (VDSL 2: up to 100Mbps)
- Cable Modem 10Gbps (DOCSIS 3.1)
- DS1/T1 1.5Mbps (24 telephone lines)
 - DS3/T3 45Mbps (28 T1's)
- E1 2Mbps (32 lines)
 - E_n 4^{n-1} E1's
- STS-1 51.840Mbps
- STS- n $n \times$ STS-1 (also called OC- n)
 - OC-192: 10Gbps
- STM- n $n \times$ STS-3; OC-768=STM-256: 40Gbps



Wireless

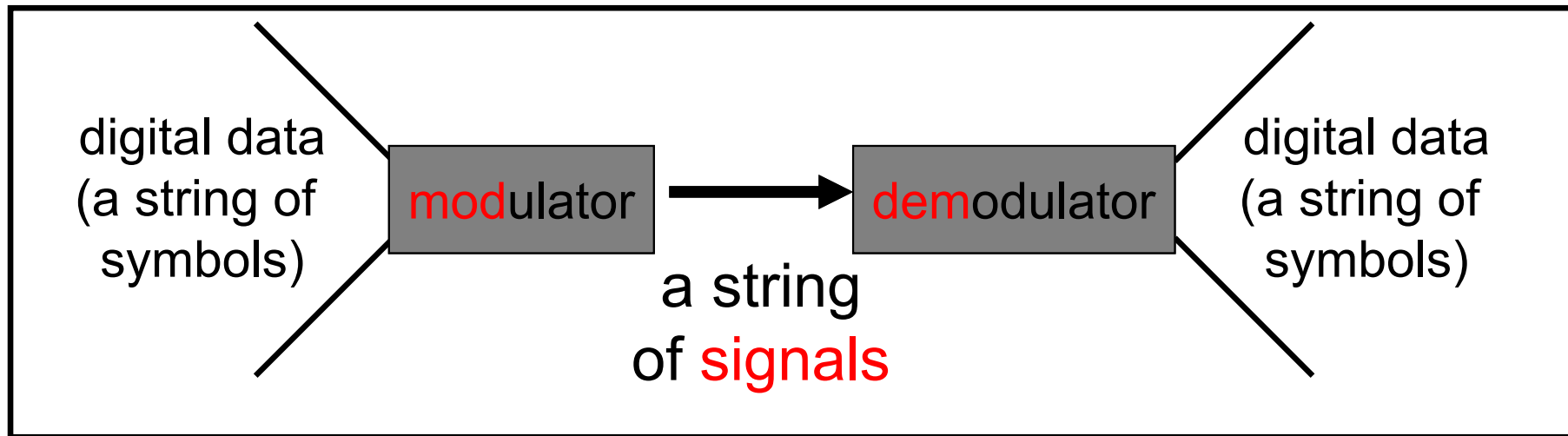
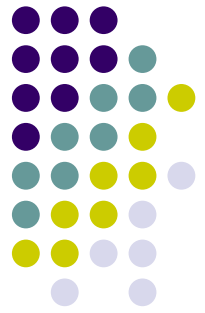
- Cellular
 - 2G->3G 64Kbps -> 2Mbps
 - 4G LTE -> 5G 1Gbps -> 10Gbps
- Wireless Local Area Networks (WLAN)
 - 2.4GHz (802.11n) 300Mbps
 - 5GHz (802.11ac) 6Gbps
 - 6Ghz (802.11ax-> be) 9Gbps -> 23Gbps
 - Bluetooth (2 and up) 3Mbps 10m
- Satellites
 - Geosynchronous 600-1000Mbps
 - Low earth orbit (LEO) ~400Mbps

Encoding

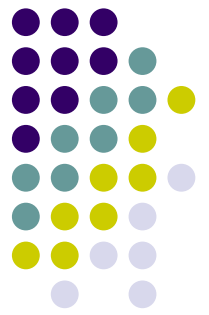


- Goal: Let physical phenomena carry information (bits)
- Idea:
 - Modulate electromagnetic waves
 - Vary amplitude, frequency, phase
 - Use physical medium to propagate signals
 - Data is encoded in the signal

Encoding



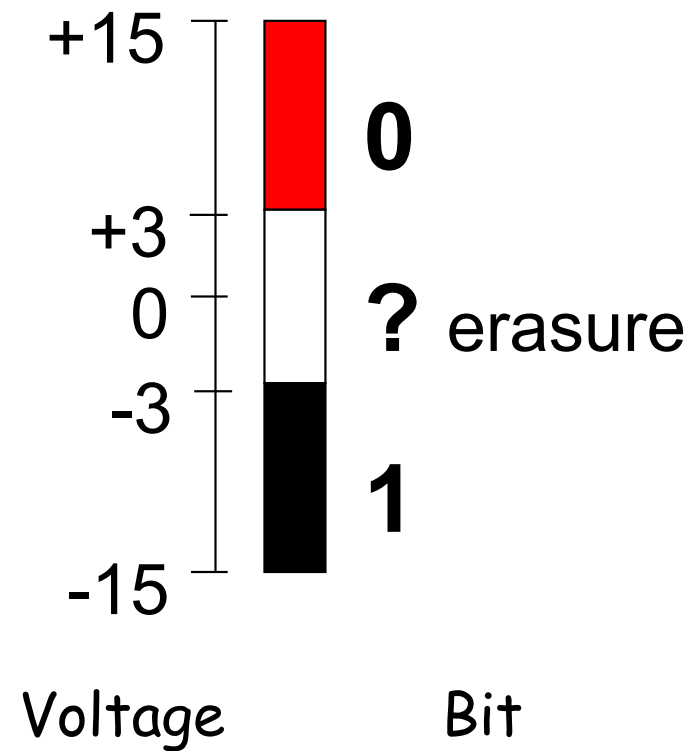
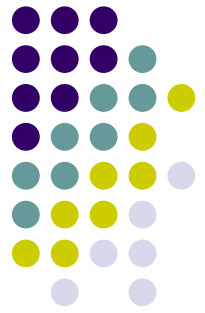
- Problems with signal transmission
 - Attenuation: Signal power absorbed by medium
 - Dispersion: A discrete signal spreads in space
 - Noise: Random background "signals"

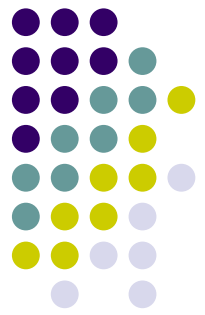


Voltage Encoding

- Common binary voltage encodings
 - Non-return to zero (NRZ)
 - Manchester (used by original Ethernet)
 - NRZ inverted (NRZI)
 - 4B/5B

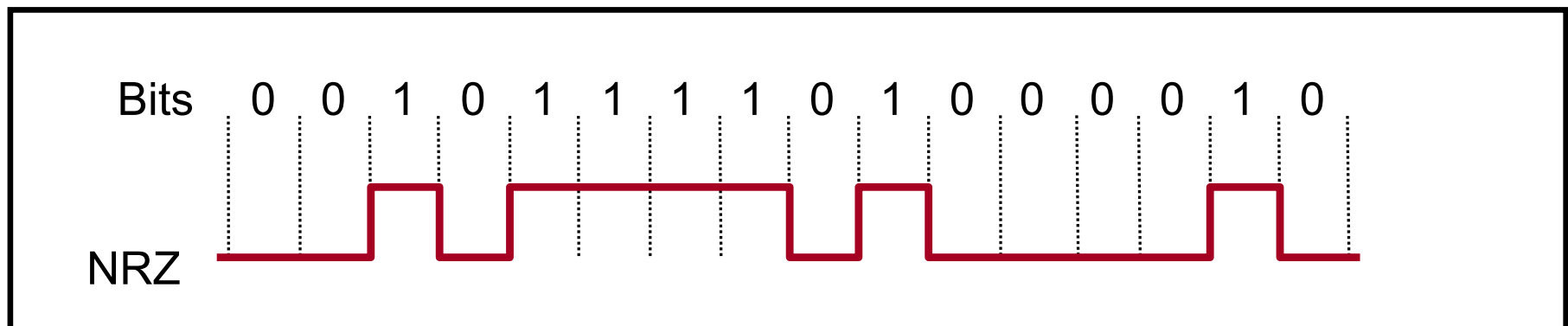
Analog Encoding: NRZ

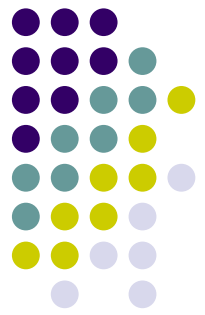




Non-Return to Zero (NRZ)

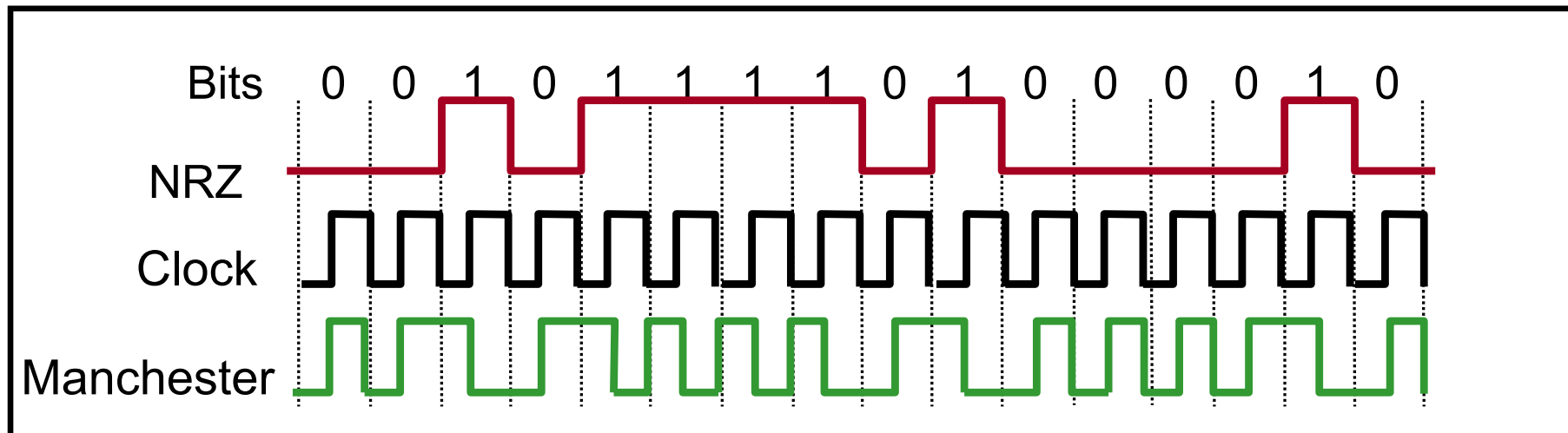
- Encoding($x_1, x_2, x_3 \dots$) = $y_1, y_2, y_3 \dots$
 - x_i are input bits
 - y_i are outputs: $y_i=1$ means high voltage/light, $y_i=0$ is low/dark
- NRZ: $y_i = x_i$
- Comment
 - Long strings of identical bits inhibit clock recovery



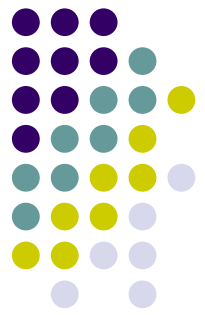


Manchester Encoding

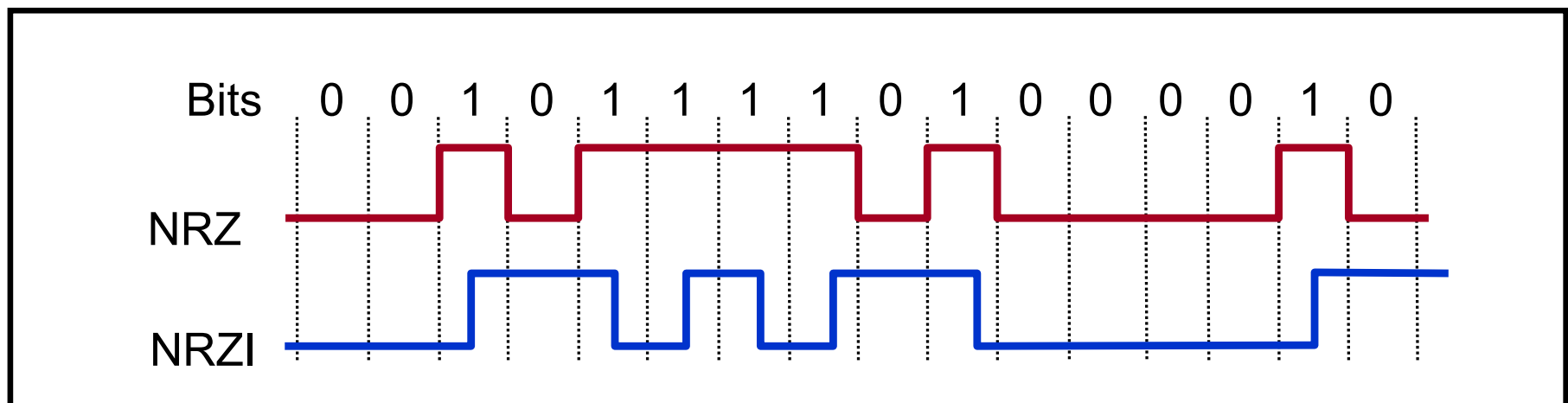
- Encoding: use double speed
 - $x_i = 1 \Rightarrow y_i = \text{hi;lo}$
 - $x_i = 0 \Rightarrow y_i = \text{lo;hi}$
- Solves clock recovery problem
 - But only 50% efficient ($\frac{1}{2}$ bit per transition)



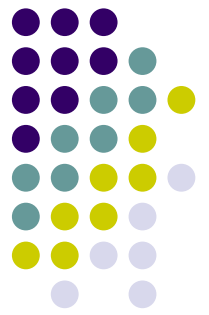
Non-Return to Zero Inverted (NRZI)



- NRZI Encoding:
 $Y_{i+1} = Y_i \oplus X_{i+1}$ (flip signal iff input bit is 1)
- Comments
 - Solves series of 1s, but not 0s

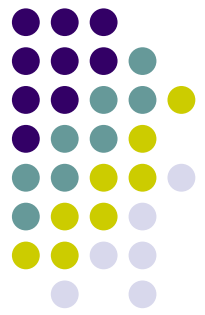


4B/5B



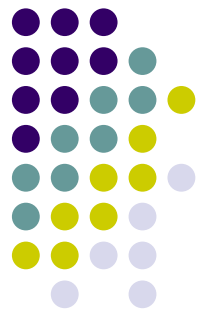
- Idea: get many transitions
 - at a price lower than 100% overhead (Manchester)
- Implementation: A special table to encode every 4 consecutive bits as a 5 bit symbol
- Properties of symbols:
 - At most 1 leading 0, at most 2 trailing 0s
 - Therefore, never more than 3 consecutive 0s!
 - Use NRZI: no problem with consecutive 1's!
- Efficiency: 80%
- Used in FDDI; additional bit sequences used with special meaning

4B5B encoding



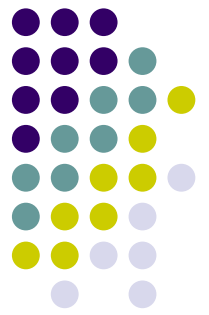
<u>4-bit symbol</u>	<u>5-bit code</u>
0000	11110
0001	01001
0010	10100
0011	10101
0100	01010
0101	01011
0110	01110
0111	01111

<u>4-bit symbol</u>	<u>5-bit code</u>
1000	10010
1001	10011
1010	10110
1011	10111
1100	11010
1101	11011
1110	11100
1111	11101



What Limits the Bit Rate?

- Rate typically limited by signaling properties.
 - No matter how small the voltage or how short the wire, changing voltages takes time.
- Electronics is slow compared to optics.
- Nyquist-Hartley-Shannon: channel with bandwidth B and additive white Gaussian noise has capacity $B \log_2(1 + S/N)$, where S/N is the **Signal to Noise Ratio**.

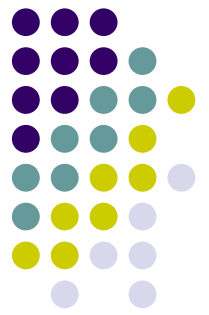


Error Detection

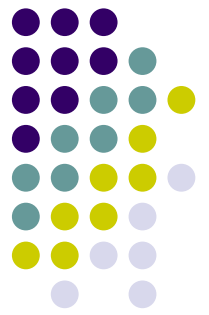
Basic Idea: Add redundancy to be clearer

- Errors checked at many levels
 - Demodulation of signals into symbols (analog)
 - Bit error detection/correction (digital)—our main focus
 - Within network adapter (CRC check)
 - Within IP layer (IP checksum)
 - Possibly within application as well

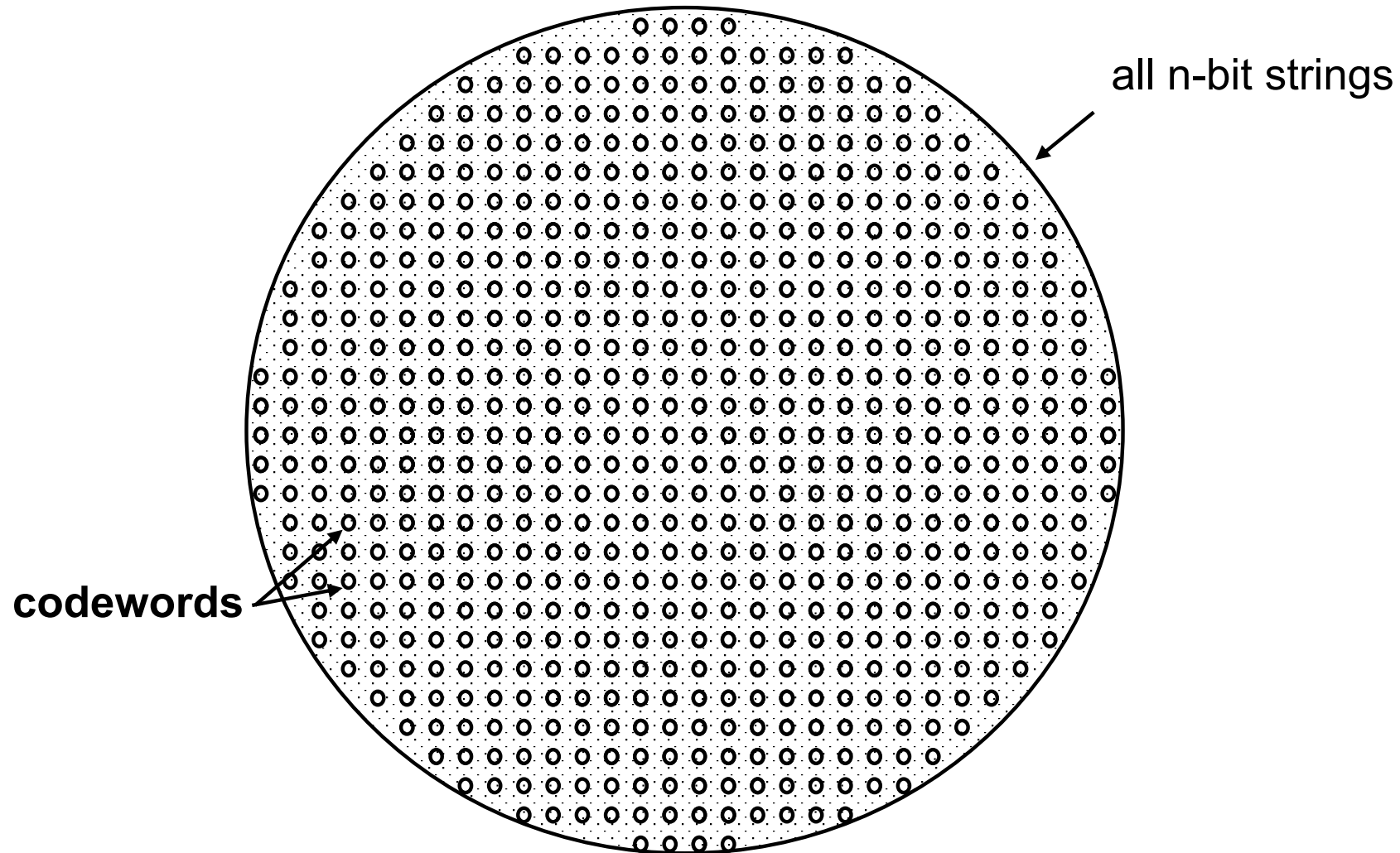
Error Detection Topics



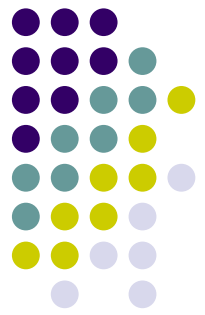
- Analog Errors
 - Example of signal distortion
- Hamming distance
 - Parity and voting
 - Hamming codes
- Error bits or error bursts?
- Digital error detection
 - Two-dimensional parity
 - Checksums
 - Cyclic Redundancy Check (CRC)



Error Detection: Intuition

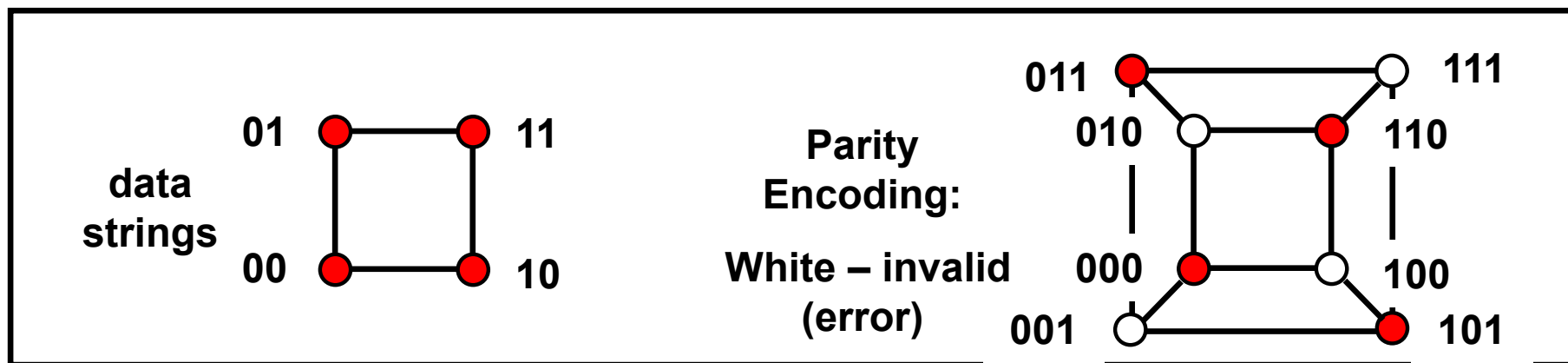


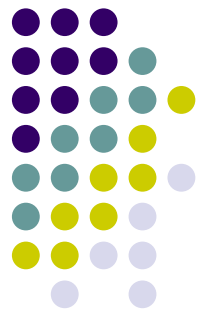
A code is a (sparse) subset of the bitstrings



Error detection: Parity

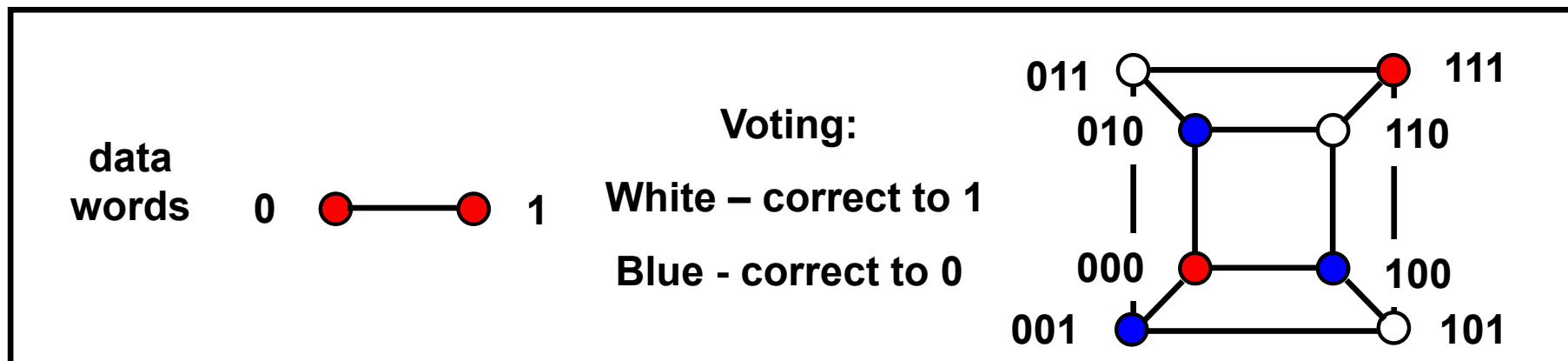
- Add an extra bit to a code to ensure an even (odd) number of 1s
 - Every code word has an even (odd) number of 1s
- Detects all 1-bit errors



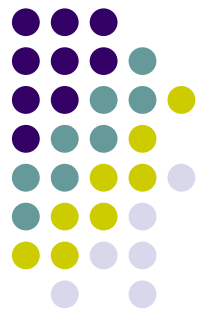


Error detection: Replication

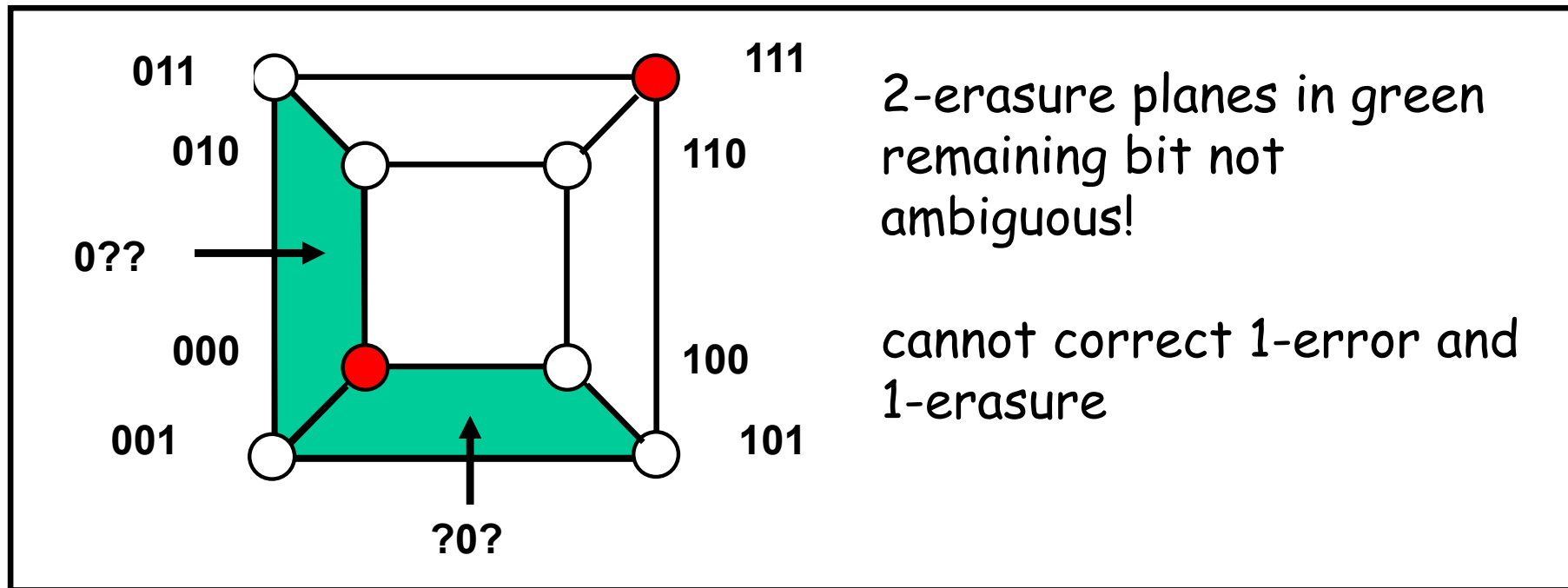
- Every bit is transmitted n times
- For $n=3$, get 1-bit error correction by voting



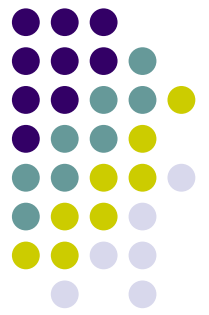
2-bit Erasure Correction with Voting



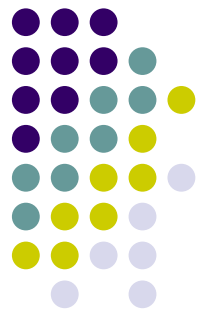
- Every code word is copied 3 times



Digital Error Detection Techniques

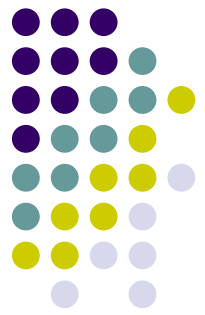


- Two-dimensional parity
 - Detects up to 3-bit errors
 - Good for burst errors
- Cyclic Redundancy Check (CRC)
 - Powerful mathematics
 - Simple in hardware
 - Used in network adapters
- (IP checksum)



Hamming Distance

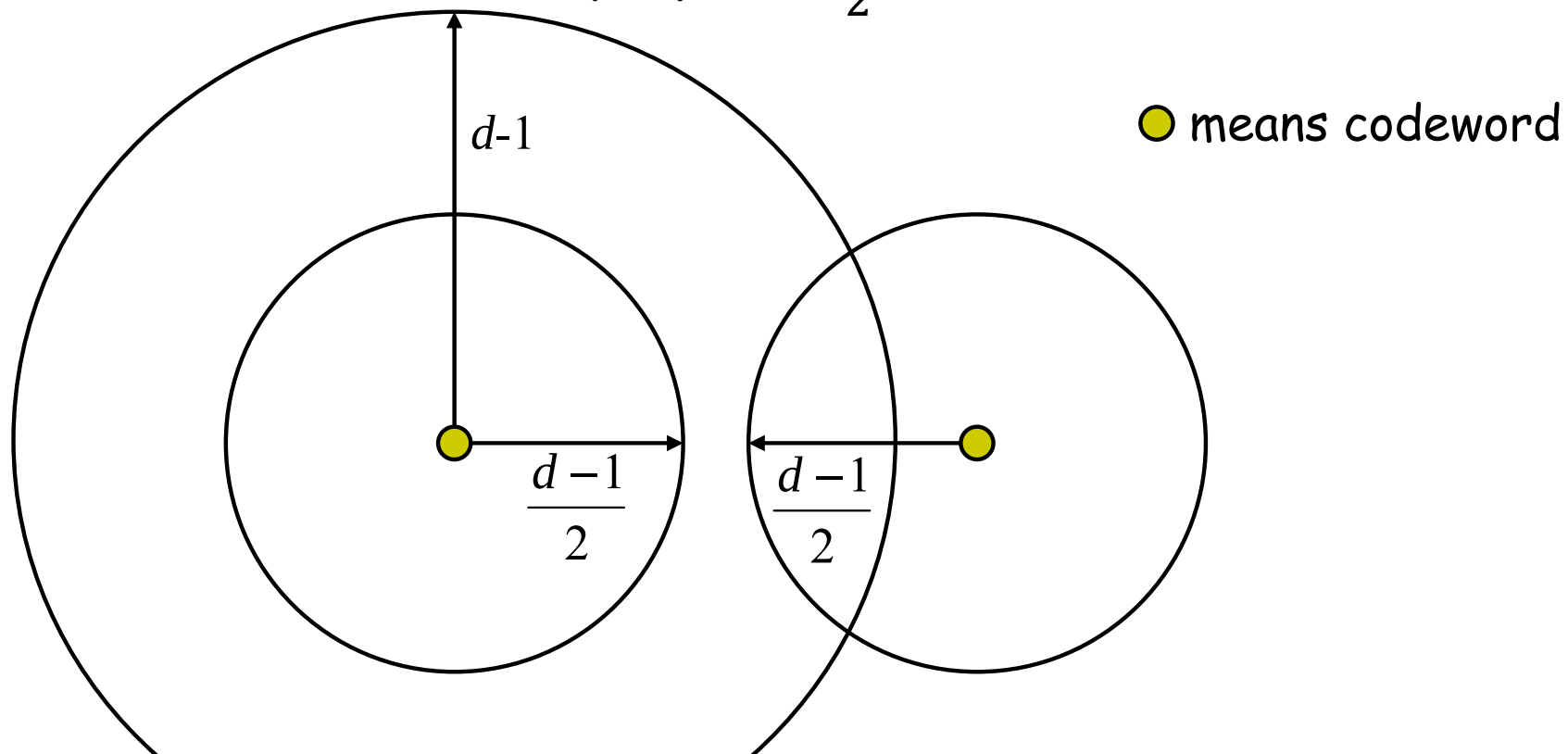
- Definition: **Hamming distance** between two equal-length strings is the number of positions in which they differ
 - Example: 00101 and 00010 have a distance of 3
- The minimum Hamming distance of a code is the minimum distance over all pairs of codewords
 - 2 for parity
 - k for k -replication



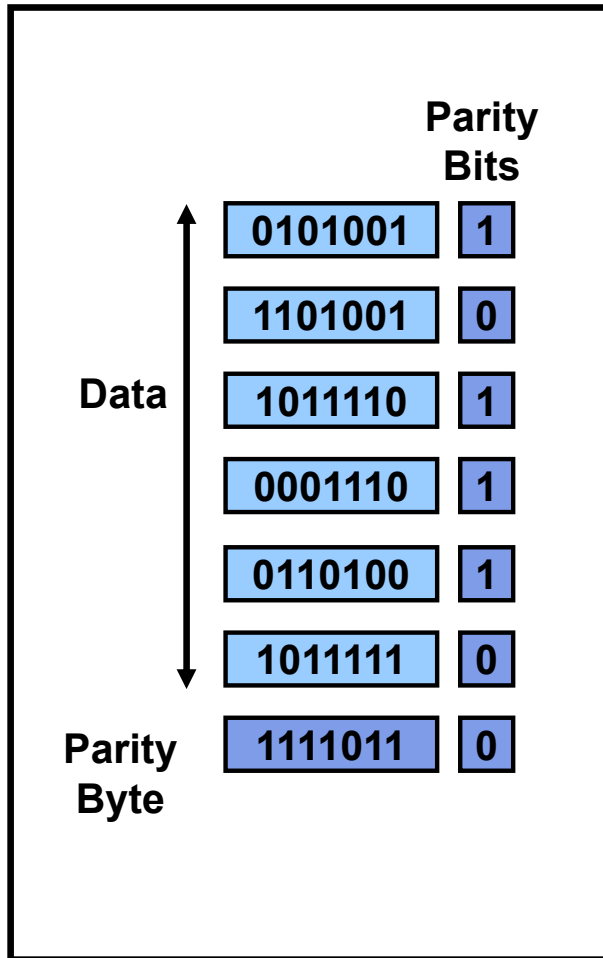
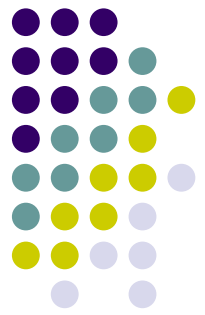
Detection and Correction

An (n, k, d) codes: n bits, k information bits, min distance d

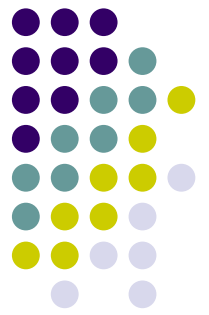
- Can **detect** any $d - 1$ errors
- Can **correct** correctly up to $\frac{d-1}{2}$ errors



Two-Dimensional Parity



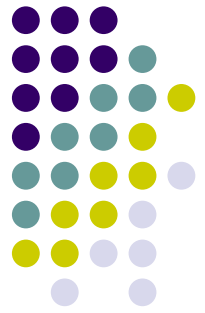
- Use 1-dimensional parity
 - Add one bit to a 7-bit code to ensure an even/odd number of 1s
- Add 2nd dimension
 - Add an extra byte to frame
 - Bits are set to ensure even/odd number of 1s in that position across all bytes in frame
- What is the 2d-code for $n \times 1$ matrix?



Hamming Codes

- 1-bit error-correcting codes
- Minimal number of check bits required
- Construction:
 - Notation: Let the binary representation of k be $k_1k_2k_3 \dots$
 - Let the codeword be $b_1b_2 \dots b_n$. Bits at position 1,2,4, ... are **check bits**, others are **data bits**
 - Value of check bit b_p : XOR of all bits b_k for k such that the p^{th} bit in the binary representation of k is set:

$$b_{2^i} = \bigoplus \{b_k \mid k_{2^i} = 1\}$$



Hamming Codes

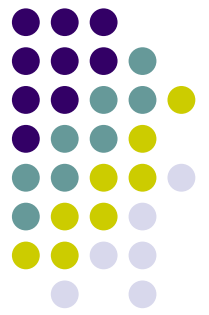
Example: 4 bits of data, 3 check bits

1	2	3	4	5	6	7
C₁	C₂	D₃	C₄	D₅	D₆	D₇
001	010	011	100	101	110	111

$$C_1 = D_3 \oplus D_5 \oplus D_7$$

$$C_2 = D_3 \oplus D_6 \oplus D_7$$

$$C_4 = D_5 \oplus D_6 \oplus D_7$$



Hamming Codes: Example

- Say the data is 0100.
- Then $D_3 = 0, D_5 = 1, D_6 = 0, D_7 = 0$.
- Therefore $C_1 = 1, C_2 = 0, C_4 = 1$
→ transmit 1001100
- Suppose we receive 1011100.
- Verify checkbits:
 - $C_1 + D_3 + D_5 + D_7 = 1$
 - Position 0 (lsb) of index
 - $C_2 + D_3 + D_6 + D_7 = 1$
 - Position 1 of index
 - $C_4 + D_5 + D_6 + D_7 = 0$
 - Position 2 of index

C_1	C_2	D_3	C_4	D_5	D_6	D_7
-------	-------	-------	-------	-------	-------	-------

$$C_1 = D_3 \oplus D_5 \oplus D_7$$

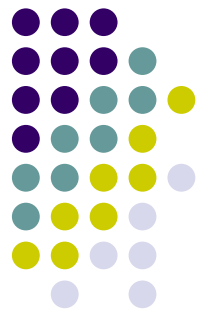
$$C_2 = D_3 \oplus D_6 \oplus D_7$$

$$C_4 = D_5 \oplus D_6 \oplus D_7$$

Result:

The error is at index

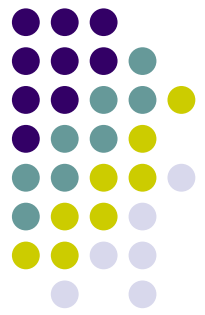
$$011_2 = 3 (!)$$



General Hamming code

- Hamming codes have parameters (n, k, d) :
 $n = 2^m - 1$, $k = 2^m - 1 - m$, $d = 3$,
for any natural $m \geq 3$.
 $(7, 4, 3)$, $(15, 11, 3)$, $(31, 26, 3)$, ...
- These are **perfect codes**: each bitstring of length $2^m - 1$ is at distance at most 1 from a codeword
 - In other codes some bitstrings are at distance more than $(d - 1)/2$ from any codeword.

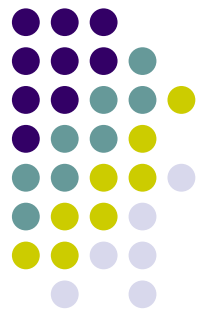
Cyclic Redundancy Check (CRC)



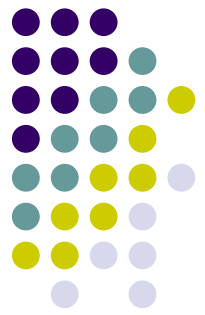
- Goal: identify runs of bit flips, cheaply
- Idea:
 - N-bit message is thought of as an N-degree polynomial: each bit in the message is a coefficient in the polynomial
 - Example:
 - Message = 10011010
 - Message polynomial:

$$\begin{aligned} M(x) &\stackrel{\text{def}}{=} 1 \cdot x^7 + 0 \cdot x^6 + 0 \cdot x^5 + 1 \cdot x^4 + 1 \cdot x^3 + 0 \cdot x^2 + 1 \cdot x + 0 \cdot 1 \\ &= x^7 + x^4 + x^3 + x \end{aligned}$$

CRC: Technique



- Select a divisor polynomial $C(x)$ with degree k
 - Example: $C(x) = x^3 + x^2 + 1$
 - The degree is $k = 3$
 - $C(x)$ should be **non-reducible**.
- Transmit a polynomial $P(x)$ that is **divisible** by $C(x)$
 - $P(x) = M(x)$ followed by k bits



Polynomial Multiplication

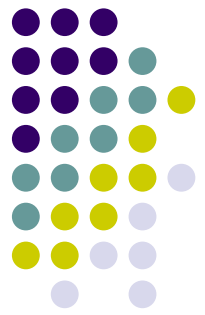
- Polynomial arithmetic in practice simply means **addition without carries**.
- Example: Multiply 1101 by 10110:
 - $x^3 + x^2 + 1$ by $x^4 + x^2 + x$

$$\begin{array}{r} 1101 \\ 10110 \\ \hline 1101 \\ 1101 \\ 1101 \\ \hline 11111110 \end{array}$$

$$x^7 + x^6 + x^5 + x^4 + x^3 + x^2 + x$$

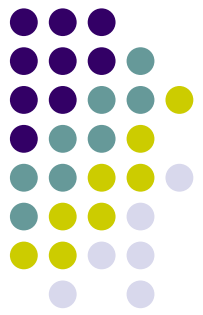
This is a multiple of $C(x)$, so that if no errors occur according to this sequence, the CRC test would be passed

Polynomial Arithmetic: Division

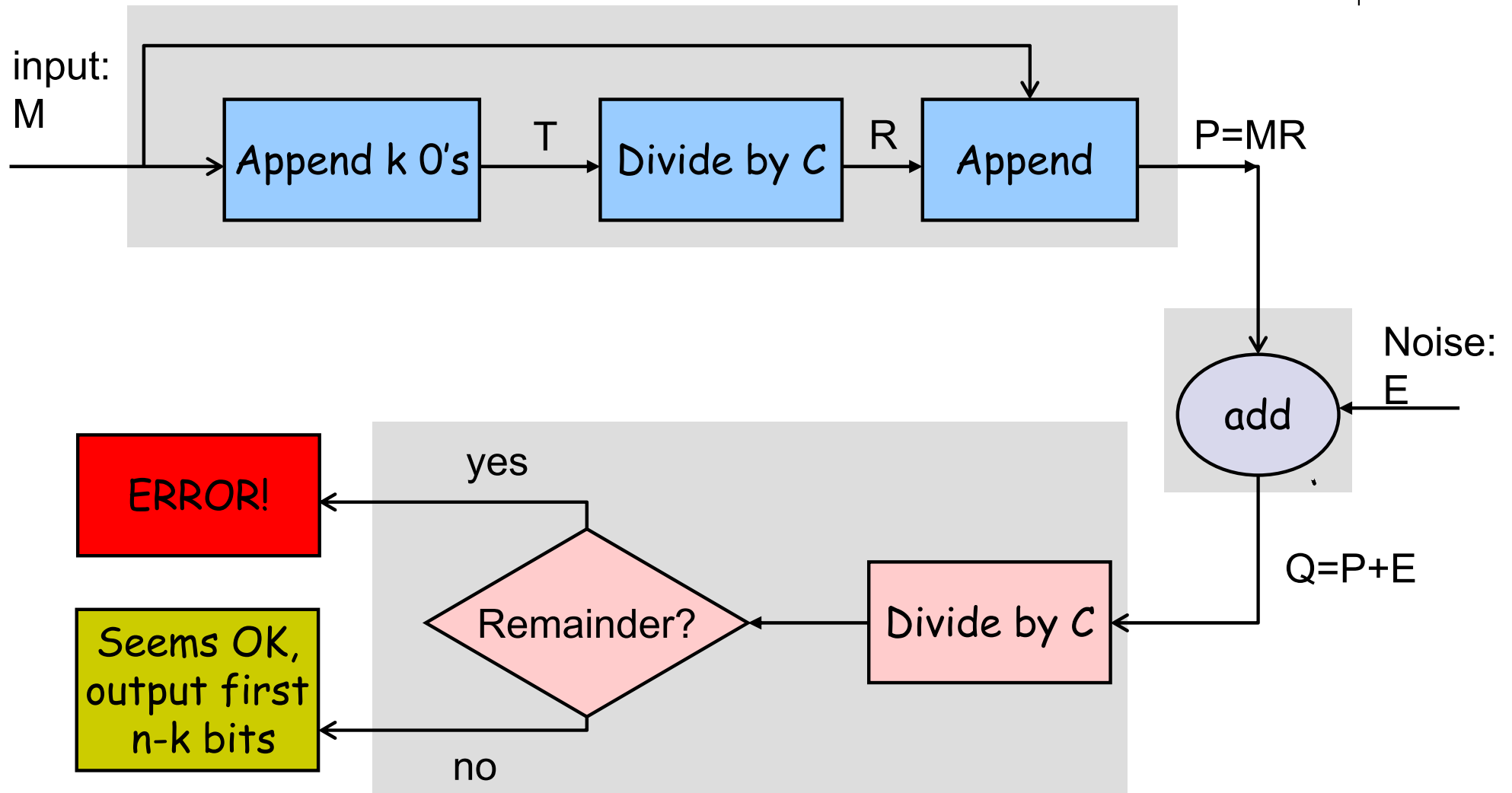


$$\begin{array}{r} \overline{10011010000} \overline{) 1101} \\ \underline{1101} \\ 1001 \\ \underline{1101} \\ 1000 \\ \underline{1101} \\ 1011 \\ \underline{1101} \\ 1100 \\ \underline{1101} \\ 1000 \\ \underline{1101} \\ 101 \end{array}$$

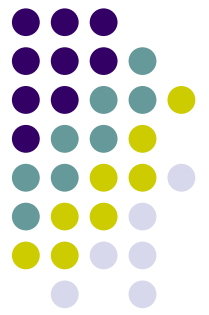
remainder!



CRC Process



CRC - Sender



- Steps

1. $T(x) = M(x) \cdot x^k$ (zero extending).
2. Find remainder, $R(x)$, from $T(x)/C(x)$
3. $P(x) = T(x) - R(x)$

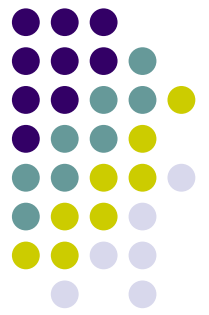
$$[T := M \ 0^k]$$

$$[R := T \bmod C]$$

$$[P := M \ R]$$

- Example (cont.)

- $M(x) = 10011010 = x^7 + x^4 + x^3 + x$
- $C(x) = 1101 = x^3 + x^2 + 1$
- $T(x) = 100110100000$
- $R(x) = 101$
- $P(x) = 10011010101$

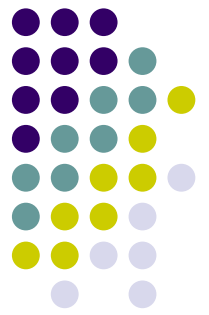


CRC - Channel

Abstract modeling of errors:

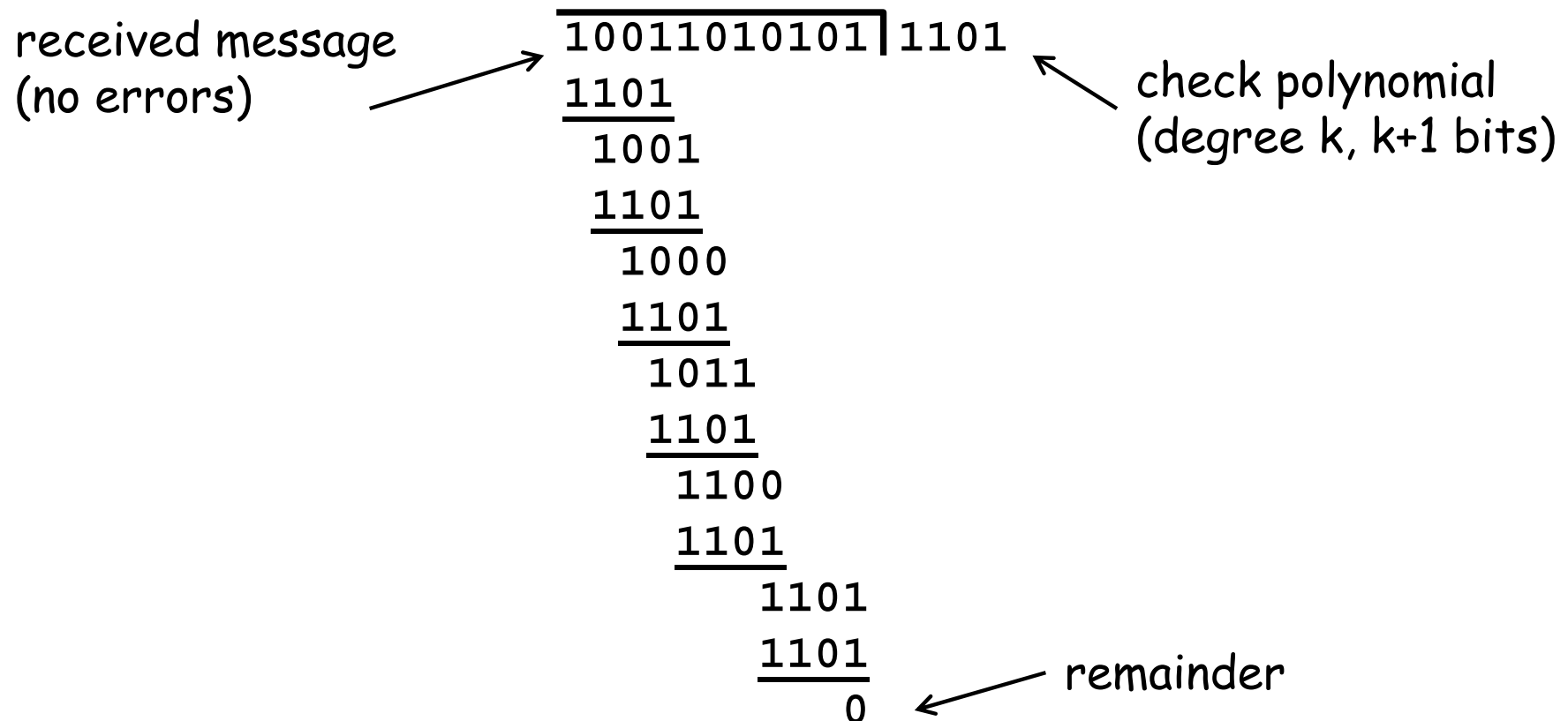
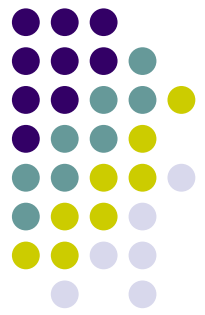
- Receive Polynomial $P(x)$ from sender
- Noise inverts some bits
- Deliver $P(x) + E(x)$ to sender
 - $E(x)$ represents errors
 - $E(x) = 0$ implies no errors

CRC - Receiver



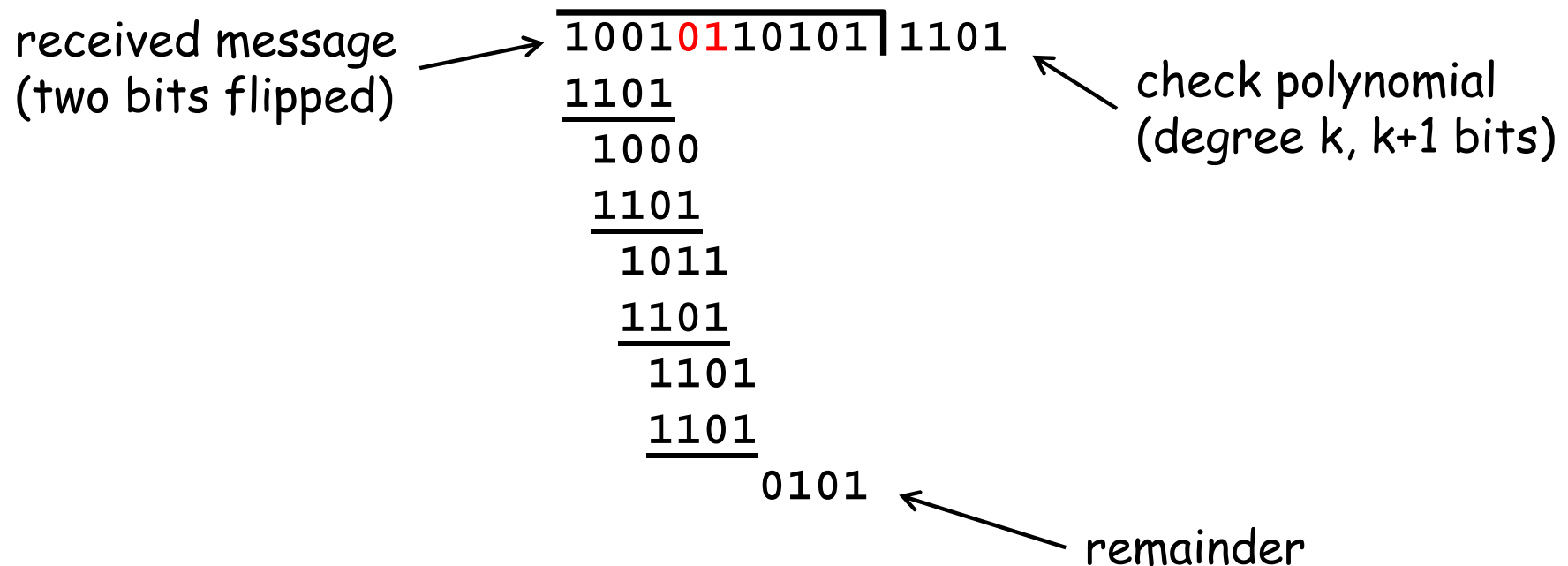
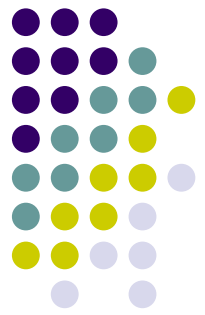
- Receive Polynomial $Q(x) = P(x) + E(x)$
- Divide $Q(x)$ by $C(x)$, find remainder
- If remainder is not 0, then definitely some errors occurred!
- If remainder is 0 then either
 - No errors ($E(x) = 0$), or, more generally...
 - $E(x)$ is divisible by $C(x)$ but may be non-zero, namely some errors may have occurred

CRC Decoding: no errors example



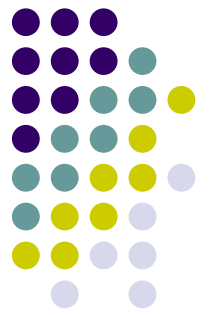
Result: remainder = 0, so CRC test **passed**

CRC Decoding: errors example



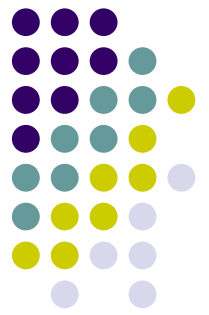
Result: remainder $\neq 0$, so CRC test **failed**

On Polynomial Arithmetic



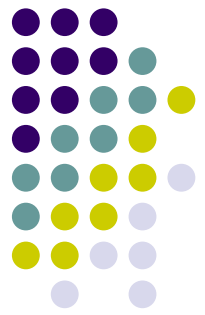
- An error is not detected if and only if the error polynomial $E(x)$ is a multiple of $C(x)$
- Example implication
 - Suppose $C(x)$ has the property that $C(1) = 0$
 - (i.e. $(x + 1)$ is a factor of $C(x)$)
 - If $E(x)$ corresponds to an undetected error pattern, then it must be that $E(1) = 0$
 - Therefore, any error pattern with an odd number of error bits is detected

CRC Error Detection



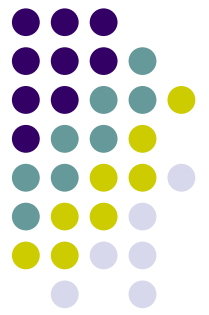
- What errors can we detect?
 - Any odd number of bit errors, if $C(x)$ contains the factor $(x + 1)$
 - All single-bit errors, if x^k and x^0 have non-zero coefficients
 - All double-bit errors, if $C(x)$ has at least three terms
 - Any burst of length less than k
 - Most bursts of length $\geq k$

Common Polynomials for $C(x)$



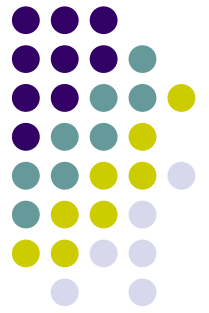
CRC	$C(x)$
CRC-8	$x^8 + x^2 + x^1 + 1$
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^1 + 1$
CRC-12	$x^{12} + x^{11} + x^3 + x^2 + x^1 + 1$
CRC-16	$x^{16} + x^{15} + x^2 + 1$
CRC-CCITT	$x^{16} + x^{12} + x^5 + 1$
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1$

Internet Checksum



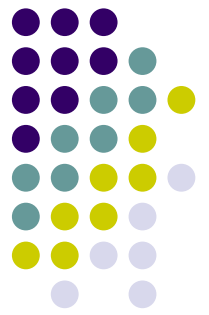
- Idea: Use 1's complement addition on 16 bit words
 - msb is sign; magnitude: remainder of the word (bits negated if msb=1). When adding, add 1 if carry overflow.
 - Example
 - Codewords: -3 -4
 - 1's complement binary: 1100 1011
 - sum: 10111 → 1's complement sum 1000 = -7
- Comments
 - Small number of redundant bits
 - Easy to implement
 - Not very robust

IP Checksum

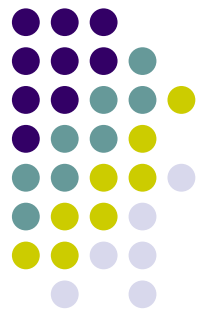


```
u_short cksum(u_short *buf, int count) {
    register u_long sum = 0;
    while (count-- > 0) {
        sum += *buf++;
        if (sum & 0xFFFF0000) {
            /* carry occurred, so wrap around */
            sum &= 0xFFFF;
            sum++;
        }
    }
    return ~(sum & 0xFFFF);
}
```


Error Detection vs. Error Correction



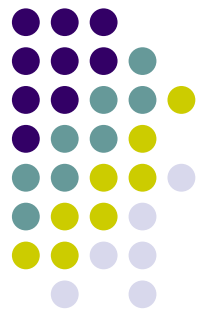
- Detection
 - Pro: Overhead only on messages with errors
 - Con: Cost in bandwidth and latency for retransmissions
- Correction (FEC: Forward Error Correction)
 - Pro: Quick recovery
 - Con: Overhead on all messages
- What should we use?
 - Correction if retransmission is too expensive
 - Correction if probability of errors is high



New Concepts

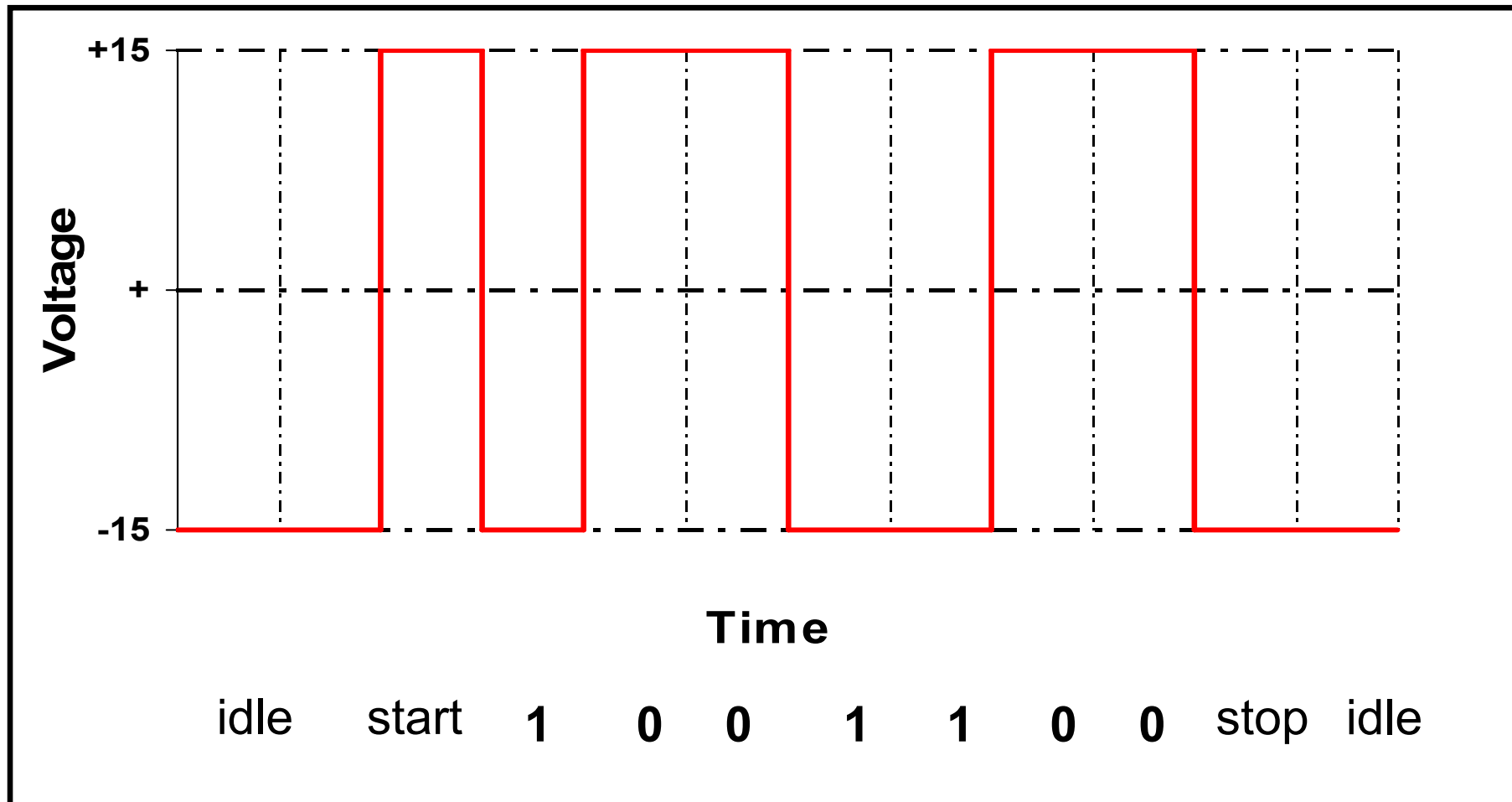
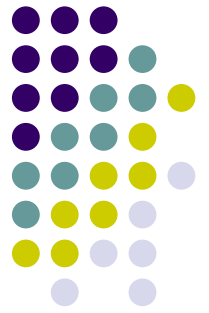
- Modem
- Twisted pair
- Multi/single mode fiber
- OC- n lines
- NRZ, NRZI
- Manchester encoding
- 4b/5b
- Hamming distance
- Error detection
- Error correction
- Parity & replication codes
- 2d-parity code
- Hamming code
- CRC codes

Example: RS-232

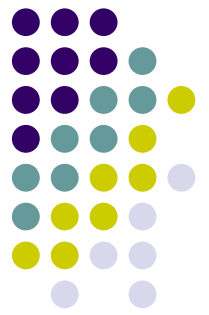


- Basic communication between computer and peripherals ("serial port")
- Modulate using two voltage levels (+15V, -15V)
- Initially slow (19.2 kbps in RS-232-C); raised later
- Characteristics
 - Serial: one signaling wire, one bit at a time
 - Asynchronous: line can be idle, clock generated from data
 - Character-based: send data in 7- or 8-bit characters

RS-232 Timing Diagram

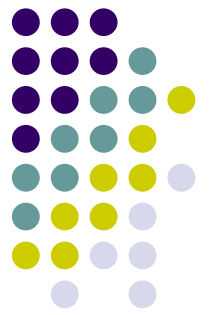


RS-232



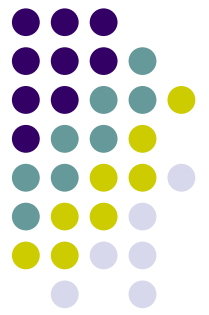
- One bit per clock
- Voltage never returns to 0V (0V means dead!)
- -15V is both idle and "1"
- Framing by start bit (15V for one clock), and stop bit (idle for one clock at -15V), for each character
- One character leads to 2+ voltage transitions
- Total of 9 bits for 7 bits of data (78% efficient)

Framing



- Demarcates units of transfer
 - Example: RS-232: start (0) and stop (1) bits, fixed length
- Goal: Allow "fields" in communication
- Need to
 - determine what set of bits constitute a frame?
 - How do we determine the beginning and end of a frame?

Framing

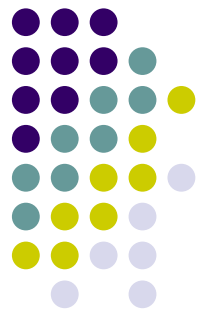


Breaks up continuous streams of bytes

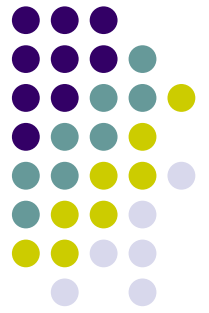
Advantages:

- Synchronization recovery
 - What bit are we talking about now?
- Efficient error detection and correction
 - Per-frame error checking and recovery
- Link multiplexing
 - Multiple hosts on shared medium
 - Simplifies multiplexing of logical channels

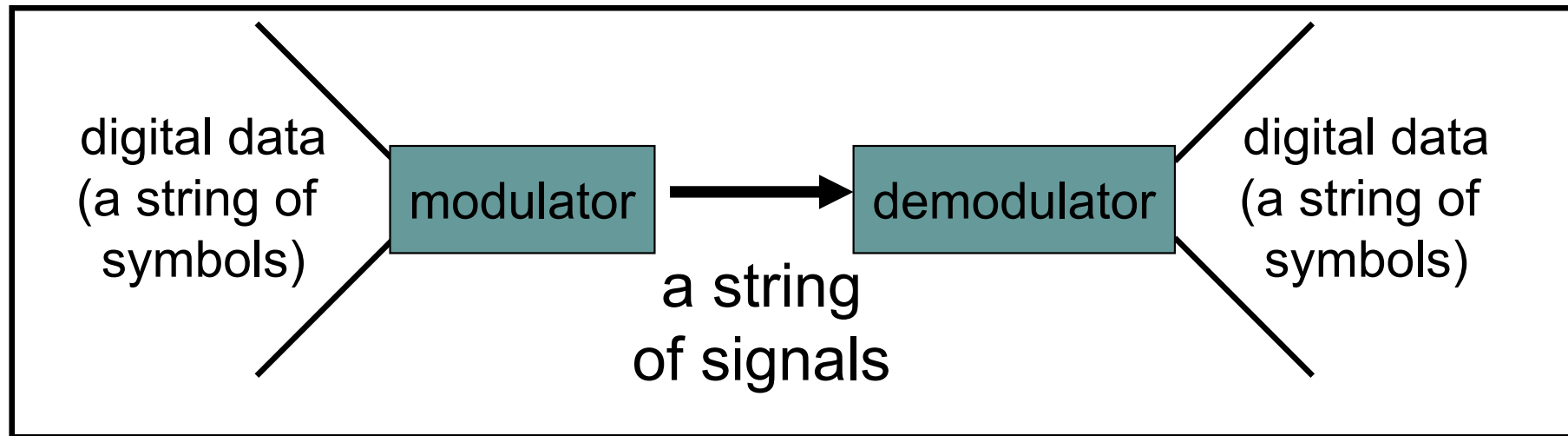
Framing



- Approaches
 - Sentinel (like C strings)
 - Length-based (like Pascal strings)
 - Clock based
- Characteristics
 - Bit- or byte-oriented
 - Fixed or variable length
 - Data-dependent or data-independent length



Error Detection



- Encoding translates symbols to signals
- Framing demarcates units of transfer
- Error detection validates correctness of each frame